

Yuhao Ge geyuhao33@gmail.com | (217) 926-3291 | [Homepage](#) | [Linkedin](#)

LLM infrastructure and performance engineer with cross-accelerator expertise (TPU, Trainium, GPU), spanning compiler optimization, kernel implementation, and large-scale serving systems.

Education

University of Illinois at Urbana-Champaign | M.S. in Computer Science, 2023/08 - 2024/12

University of Illinois at Urbana-Champaign | B.S. in Computer Engineering, GPA 4.0/4.0 2019/08 - 2023/05

– Honors: Highest Honors, Bronze Tablet (3%, 2023), Dean's List (2020 & 2022)

Skills

Programming Languages: Python, C/C++, CUDA, SystemVerilog, Assembly

ML Framework & Compiler: PyTorch, JAX, Triton, Pallas, vLLM, DeepSpeed, ZeRO/FSDP, XLA, MLIR, TVM

Experience

Google | Software Engineer | *TPU, Gemini, PyTorch, vLLM, Pallas* 2025/11 - Present

- Optimize Gemini training and serving performance on TPU through hardware-aware quantization.
- Develop the first PyTorch-native vLLM TPU backend, enabling efficient serving of arbitrary PyTorch models on TPU.
- Implement and optimize Pallas kernels for performance-critical workloads, including Attention and MoE.
- Optimize large-scale open-source LLM workloads (Qwen, GPT-OSS) within the vLLM TPU backend.

AWS, Annapurna Labs | Software Engineer | *ML Compiler, LLMs, LLM Systems, Architecture* 2025/03 - 2025/11

- Founding engineer of the [NKI](#) kernel language and compiler for AWS [Trainium](#), driving language and compiler design.
- Architected and optimized core kernel compiler passes (scheduling, allocation) with cycle-level optimization.
- Designed and implemented custom FlashAttention kernels tailored for LLM workloads on Trainium.
- Led development of autotuning infrastructure to systematically optimize compiler and kernel performance.

AWS, Annapurna Labs | Software Engineer Intern | *ML Compiler, LLM Systems* 2024/05 - 2024/08

- Developed automated kernel generation and profiling infrastructure to collect micro-architectural performance data and train ML-based cost models for instruction latency estimation.
- Architected the first-generation autotuning infrastructure from scratch, designing a modular, pass-agnostic framework with clean separation of design space, search strategy, and surrogate cost modeling.
- Used autotuning to optimize the Matmul Fusion Pass, achieving a 14.7% improvement for the Llama3.1 model.
- Implemented multi-process compilation and distributed profiling, accelerating autotuning workflows by 8.6x.
- Recognized with a [Certificate of Appreciation](#) for high-impact contributions.

TikTok | Software Engineer Intern | *ML, Graphics, Game Engine, AR/VR.* 2022/05 - 2022/08

- Developed TikTok's 3D Game Engine, which empowers users to create/use interactive AR/VR stickers.
- Implemented a query-based Motion Matching system in C++ for real-time, low-latency avatar control.
- Developed an SDK for Skeleton Retargeting, supporting animation adaptation across character models

University of California, Los Angeles | Research Assistant | *ML/RL, FPGA* 2022/06 - 2022/11

- Developed a GNN-based surrogate cost model to approximate HLS scheduling and resource estimation.
- Designed an ML-driven design space exploration framework for automated FPGA accelerator optimization.
- Integrated reinforcement learning and bandit strategies for adaptive search, improving exploration efficiency by 11%.

Publications

- [SPLAT](#): A framework for optimised GPU code-generation for SParse reguLar ATtention (OOPSLA' 25).

Projects

Optimized GPU code generation framework for SParse reguLar Attention 2023/08 - 2024/07

- Developed SPLAT, an optimized framework for efficient sparse-MHSA, targeting moderate sparsity levels.
- Introduced Affine Compressed Sparse-Row (ACSR) format for regular sparsity patterns in MHSA.
- Engineered advanced GPU code-generation algorithms for ACSR, enhancing sparse-MHSA kernel performance.
- Achieved 2.05x and 4.05x speedups over Triton and TVM. Accepted at OOPSLA 2025.

Implement A “Doodle Jump” Efficiently on the FPGA Board 2022/01 - 2022/05

- Ported the game “Doodle Jump” to FPGA with SystemVerilog, achieving low power consumption and high efficiency.
- Implemented a SOC with NIOS II in C to manage complex tasks like USB protocol and memory I/O.
- Consumed only 400KB memory, 0.5w power to achieve a 50 HZ frame rate, won the *Best Design Prize*.